

APOLLO INSTITUTE OF COMPUTER APPLICATION

BACHELOR OF COMPUTER APPLICATION

SEMESTER 03

Object Oriented Programming with Java (BC03001021)

Lecture Notes

Unit 1: Introduction to Java (4 Hours)

Lecture	Topics Covered
1	Basics of Java, Background/History of Java, Java and the Internet
2	Advantages of Java, JVM, JRE, JDK, Bytecode
3	Java Environment Setup, Java Program Structure, POP vs OOP
4	OOP Concepts (Abstraction, Encapsulation, Inheritance, Polymorphism, Overloading, Message Communication), Hello World Program, Compilation & Common Errors

Lecture – 1

1.1 Basics of Java, Background/History of Java, Java and the Internet

Learning Outcomes

- Define Java.
 - Explain the history and evolution of Java.
 - Describe why Java was developed.
 - Explain the role of Java in Internet applications.
 - Identify the major applications of Java.
-

1.1 Introduction to Java

Definition

Java is a **high-level, object-oriented, class-based, platform-independent programming language** developed by **Sun Microsystems** in **1995**. It is designed to create secure, reliable, and portable applications that can run on different operating systems without modifying the source code.

Java follows the principle:

Write Once, Run Anywhere (WORA)

This means a Java program is written once, compiled into **bytecode**, and can run on any system that has a **Java Virtual Machine (JVM)**.

1.2 What is a Programming Language?

A **programming language** is a language used to write instructions that a computer can understand and execute.

Examples of programming languages include:

- C
 - C++
 - Java
 - Python
 - JavaScript
-

1.3 Why is Java Popular?

Java is one of the world's most popular programming languages because it is:

- Easy to learn
- Object-oriented
- Secure
- Platform independent
- Robust
- Widely used in industries

Today Java is used in:

- Banking Systems
 - E-Commerce Websites
 - Android Applications
 - Enterprise Software
 - Cloud Computing
 - Web Applications
-

Real-Life Example

Suppose you prepare a PowerPoint presentation on Windows.

The same presentation can be opened on:

- Windows
- Linux
- macOS

Similarly,

A Java program can run on different operating systems without rewriting the program.

This feature is called **Platform Independence**.

1.4 Why was Java Developed?

Before Java, programmers mainly used **C** and **C++**.

Although these languages were powerful, they had several limitations.

Problems in C and C++

- Platform dependent
- Manual memory management
- Pointer-related security issues
- Difficult Internet programming
- Lack of portability

To overcome these problems, Sun Microsystems developed Java.

1.5 Objectives of Java

The main objectives behind Java development were:

- Platform Independence
 - Security
 - Simplicity
 - Reliability
 - Robustness
 - Distributed Computing
 - Network Programming
 - Object-Oriented Programming
-

1.6 Background and History of Java

The Green Project (1991)

In **1991**, Sun Microsystems started a project known as the **Green Project**.

The aim was to develop software for electronic devices such as:

- Television
- Set-top boxes
- Remote controls
- Home appliances

The project was led by **James Gosling**, along with **Mike Sheridan** and **Patrick Naughton**.

Why was the Language Named Oak?

Initially, the programming language was named **Oak** because there was an **Oak tree** outside James Gosling's office.

However, the name **Oak** was already registered by another company.

Therefore, a new name was required.

Why was it Named Java?

During discussions, the development team selected the name **Java**, inspired by **Java coffee**, which they frequently drank while working.

The name was chosen because it was:

- Short
 - Easy to pronounce
 - Unique
 - Legally available
-

First Public Release

Java was officially introduced in **1995** by Sun Microsystems.

Its slogan became:

Write Once, Run Anywhere (WORA)

Evolution of Java

Version	Year	Major Features
Java 1.0	1995	First Release
Java 1.2	1998	Swing, Collections Framework
Java 5	2004	Generics, Enhanced for Loop
Java 8	2014	Lambda Expressions, Stream API
Java 11	2018	Long-Term Support (LTS)
Java 17	2021	LTS Version
Java 21	2023	Latest LTS Version

James Gosling – Father of Java

James Gosling is known as the **Father of Java**.

His major contributions include:

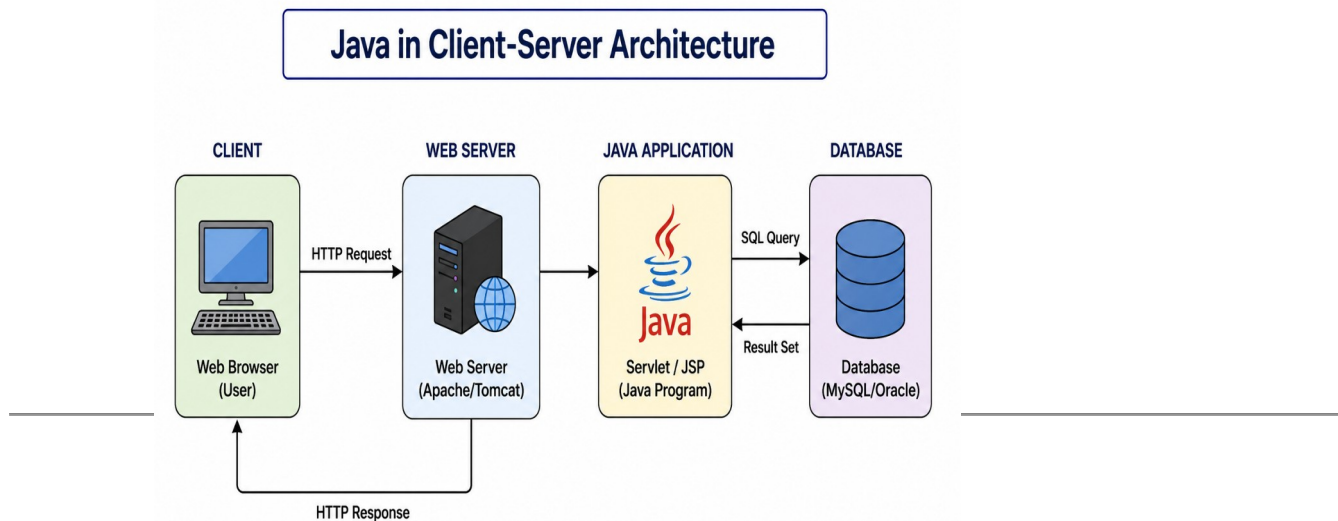
- Designed Java programming language
 - Developed the first Java compiler
 - Led the Green Project
-

1.7 Java and the Internet

Introduction

Java was designed with **Internet programming** in mind. During the early 1990s, there was a growing need for software that could run on different computers connected through the Internet.

Java solved this problem by allowing the same program to run on multiple operating systems using the **Java Virtual Machine (JVM)**.



Step 1: Client (Web Browser)

- The **Client** is the user who accesses the application through a web browser such as Chrome, Firefox, or Edge.
- The user performs an action, such as clicking the **Login** button or searching for a product.
- The browser sends an **HTTP Request** to the web server.

Example: A student enters a username and password on the college portal and clicks **Login**.

Step 2: Web Server

- The **Web Server** (such as Apache Tomcat) receives the HTTP request from the browser.
- It identifies that the request needs Java processing.
- The request is forwarded to the Java application.

Role of Web Server:

- Accepts client requests.
 - Sends requests to the Java application.
 - Returns the final response to the client.
-

Step 3: Java Application (Servlet/JSP)

- The **Java Application** contains the business logic of the system.
- It processes the user's request.
- If data is required, it communicates with the database.

Example:

For a login request, the Java application:

- Receives the username and password.
 - Checks whether the credentials are valid.
 - Requests the corresponding data from the database.
-

Step 4: Database

- The **Database** stores all application data.
- It receives queries from the Java application.
- It returns the requested information.

Examples of databases:

- MySQL

- Oracle
- PostgreSQL

Example:

The database checks whether the student's username and password exist and returns the result.

Step 5: Java Application Processes the Result

- After receiving data from the database, the Java application processes it.
- It prepares an appropriate response.

Example:

- If the login is successful, it prepares the student's dashboard.
 - If the login fails, it prepares an "Invalid Username or Password" message.
-

Step 6: Response to the Client

- The Java application sends the response to the **Web Server**.
- The Web Server sends an **HTTP Response** back to the client's browser.
- The browser displays the requested web page.

1.8 Why is Java Suitable for Internet Applications?

1. Platform Independence

Applications run on Windows, Linux, and macOS.

2. Security

Java programs execute inside the JVM.

Java provides:

- Bytecode verification
 - No direct memory access through pointers
 - Automatic memory management
-

3. Network Support

Java provides built-in packages for network programming.

Example:

- Socket Programming
 - URL Handling
 - HTTP Communication
-

4. Distributed Computing

Java supports applications running on multiple computers connected through a network.

Examples:

- Banking systems
 - Cloud applications
 - Online shopping websites
-

5. Multithreading

Java can execute multiple tasks simultaneously.

Example:

While downloading a file, you can continue browsing the Internet.

6. Applications of Java

Java is widely used in various fields.

Desktop Applications

Examples:

- Calculator
 - Text Editor
 - Inventory Management System
-

7. Web Applications

Examples:

- Online Banking
- E-Commerce Websites
- Student Management Systems

Technologies:

- Servlet
 - JSP
 - Spring Framework
-

8. Mobile Applications

Java has been widely used for Android application development.

Examples:

- WhatsApp
 - PhonePe
 - Gmail
-

Enterprise Applications

9. Large organizations use Java for:

- ERP Systems
- CRM Software

- Payroll Management

10. Cloud Applications

Java is commonly used in cloud-based services due to its scalability and reliability.

11. Scientific Applications

- Simulations
- Data Analysis
- Research Software

Key Terms

Term	Meaning
Java	High-level, object-oriented programming language
WORA	Write Once, Run Anywhere
Green Project	Initial project that led to Java
Oak	Original name of Java
JVM	Java Virtual Machine
Servlet	Java technology for server-side web applications
JSP	Java Server Pages

Summary

- Java is a high-level, object-oriented, and platform-independent programming language.
- It was developed by Sun Microsystems in 1995.
- James Gosling is known as the Father of Java.
- The language was first called **Oak** and later renamed **Java**.
- Java follows the **Write Once, Run Anywhere (WORA)** principle.
- Java is widely used for Internet, desktop, enterprise, mobile, and cloud applications.
- Its security, portability, and platform independence make it an ideal language for network-based applications.

Lecture – 2

Advantages of Java, JVM, JRE, JDK and Bytecode

Learning Outcomes

- Explain the advantages of Java.
 - Understand the architecture of Java.
 - Differentiate between JDK, JRE, and JVM.
 - Explain the concept of Bytecode.
 - Describe how a Java program is compiled and executed.
-

2.1 Advantages of Java

Java is one of the most widely used programming languages because of its powerful features. It was designed to overcome the limitations of earlier programming languages such as C and C++.

1. Platform Independent

Java follows the principle:

Write Once, Run Anywhere (WORA)

A Java program is compiled only once and can run on any operating system that has a **Java Virtual Machine (JVM)**.

Example

A Java application developed on Windows can also run on Linux or macOS without changing the source code.

2. Object-Oriented

Java is based on the **Object-Oriented Programming (OOP)** concept.

It organizes programs into **classes** and **objects**, making the code easier to understand and reuse.

Example

A **Student** object contains data such as:

- Roll Number
- Name
- Course

and methods like:

- display()
 - calculateResult()
-

3. Simple

Java syntax is simple and easy to learn.

It removes many complex features of C++ such as:

- Pointers
- Multiple inheritance through classes
- Operator overloading

Example

```
System.out.println("Welcome to Java");
```

4. Secure

Java provides high security because:

- No direct memory access through pointers
- Bytecode verification
- Automatic memory management
- JVM executes programs in a controlled environment

Real-Life Example

Just like a **bank locker** protects valuable items, the JVM protects Java programs from unsafe operations.

5. Robust

A robust language can handle errors efficiently.

Java provides:

- Exception Handling
- Garbage Collection
- Strong type checking

These features reduce program crashes.

6. Portable

Java programs can be moved from one computer to another without modification.

Example

A Java application copied from one computer to another will run correctly if the JVM is installed.

7. Multithreading

Java allows multiple tasks to execute simultaneously.

Example

While downloading a file, you can still browse the Internet.

8. Distributed

Java supports distributed applications where different computers communicate through a network.

Examples

- Online Banking

- ATM Network
 - Railway Reservation System
-

9. High Performance

Java uses the **Just-In-Time (JIT) Compiler**, which converts frequently used bytecode into machine code during execution, improving performance.

10. Dynamic

Java supports dynamic loading of classes during runtime.

Programs can load required classes whenever needed.

Summary of Advantages

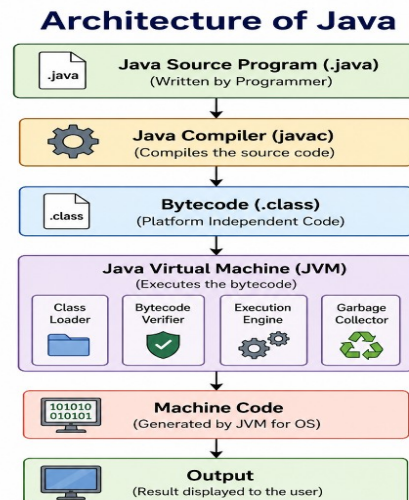
Advantage	Description
Platform Independent	Runs on any operating system
Object-Oriented	Uses classes and objects
Secure	No pointers, JVM protection
Portable	Easily transferable
Robust	Reliable with exception handling
Multithreading	Multiple tasks simultaneously
Distributed	Supports network applications
High Performance	Uses JIT Compiler
Dynamic	Loads classes at runtime

Java Architecture

Definition

Java Architecture is the overall structure of the Java platform that describes how Java programs are written, compiled, and executed. It consists of **JDK, JRE, JVM, Java API, Operating System, and Hardware**. Java architecture makes Java **platform-independent**, allowing the same program to run on different operating systems.

Overall Java Architecture Diagram



Explanation

1. Java Source Code (.java)

Definition

Java source code is the program written by the programmer using the Java programming language. The source code is saved with the **.java** extension.

Diagram

Programmer

|



Hello.java

(Source Code)

Explanation

- The programmer writes the program using any text editor or IDE such as Eclipse, NetBeans, or IntelliJ IDEA.
- Java source code contains classes, methods, variables, and objects.
- The source code cannot be executed directly by the computer.
- It must first be compiled into bytecode using the Java compiler.

Example

```
class Hello

{

    public static void main(String args[])

    {

        System.out.println("Hello World");

    }

}
```

2. Java Compiler (javac)

Definition

The **Java Compiler (javac)** is a tool that converts Java source code into **bytecode**.

Diagram

Hello.java

|



Java Compiler

(javac)

|



Hello.class

(Bytecode)

Explanation

The Java compiler performs the following tasks:

- Reads the .java source file.

- Checks syntax and semantic errors.
- Converts source code into bytecode.
- Generates a .class file.

The generated bytecode is platform-independent and can run on any computer that has a JVM.

3. Bytecode

Definition

Bytecode is the intermediate code produced by the Java compiler. It is stored in a **.class** file.

Diagram

Source Code

|



Compiler

|



Bytecode (.class)

|



JVM

Explanation

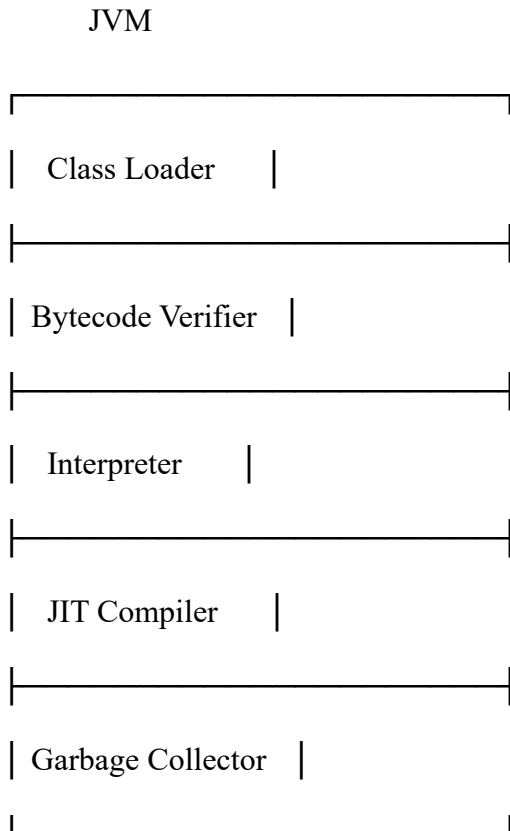
- Bytecode is neither source code nor machine code.
 - It is an intermediate language understood only by the JVM.
 - Since bytecode is platform-independent, the same .class file can run on Windows, Linux, or macOS without modification.
 - This is the main reason Java supports **"Write Once, Run Anywhere (WORA)."**
-

4. Java Virtual Machine (JVM)

Definition

The **Java Virtual Machine (JVM)** is the core component of Java Architecture. It executes Java bytecode by converting it into machine code that the operating system can understand.

Diagram



Explanation

The JVM performs several important tasks:

- Loads class files into memory.
- Verifies bytecode before execution.
- Converts bytecode into machine code.
- Executes Java programs.
- Manages memory.
- Removes unused objects using Garbage Collection.

Each operating system has its own JVM implementation, but all JVMs execute the same Java bytecode.

5. Class Loader

Definition

The **Class Loader** is a part of the JVM that loads Java class files into memory whenever they are needed.

Diagram

.class File



Class Loader



Main Memory

Explanation

- Loads classes dynamically during execution.
- Loads only the required classes.
- Saves memory by avoiding unnecessary loading.
- Supports dynamic class loading.

6. Bytecode Verifier

Definition

The **Bytecode Verifier** checks whether the bytecode is valid and secure before execution.

Diagram

Bytecode



Bytecode Verifier

|



Verified Bytecode

Explanation

The verifier checks:

- Invalid instructions
- Illegal memory access
- Stack overflow conditions
- Security violations

Only verified bytecode is allowed to execute.

7. Interpreter

Definition

The **Interpreter** reads bytecode instruction by instruction and converts it into machine instructions.

Diagram

Bytecode

|



Interpreter

|



Machine Code

Explanation

- Executes one instruction at a time.
- Suitable for small programs.

- Slower because translation occurs during execution.

8. JIT (Just-In-Time) Compiler

Definition

The **JIT Compiler** improves the performance of Java programs by converting frequently executed bytecode into native machine code.

Diagram

Bytecode

|



JIT Compiler

|



Machine Code

|



Fast Execution

Explanation

- Detects frequently used code.
- Converts it into machine code only once.
- Stores the compiled code in memory.
- Future executions become much faster.

9. Garbage Collector

Definition

The **Garbage Collector (GC)** automatically removes unused objects from memory.

Diagram

Memory

|

|— Used Objects

|— Unused Objects

|

▼

Garbage Collector

|

▼

Free Memory

Explanation

- Identifies objects that are no longer needed.
- Frees occupied memory automatically.
- Prevents memory leaks.
- Improves application performance.
- Programmers do not need to free memory manually.

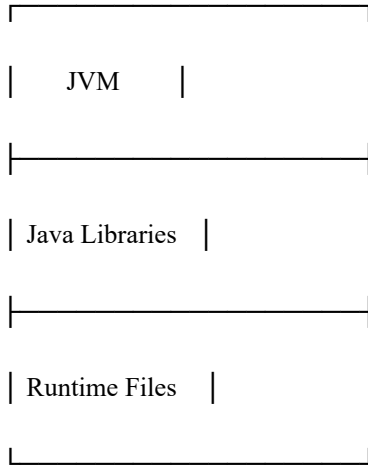
10. Java Runtime Environment (JRE)

Definition

JRE (Java Runtime Environment) provides all the software components required to run Java applications.

Diagram

JRE



Explanation

JRE contains:

- Java Virtual Machine (JVM)
- Java API (Libraries)
- Runtime support files

Its main function is to provide the environment required to execute Java programs. It does **not** contain development tools like the Java compiler.

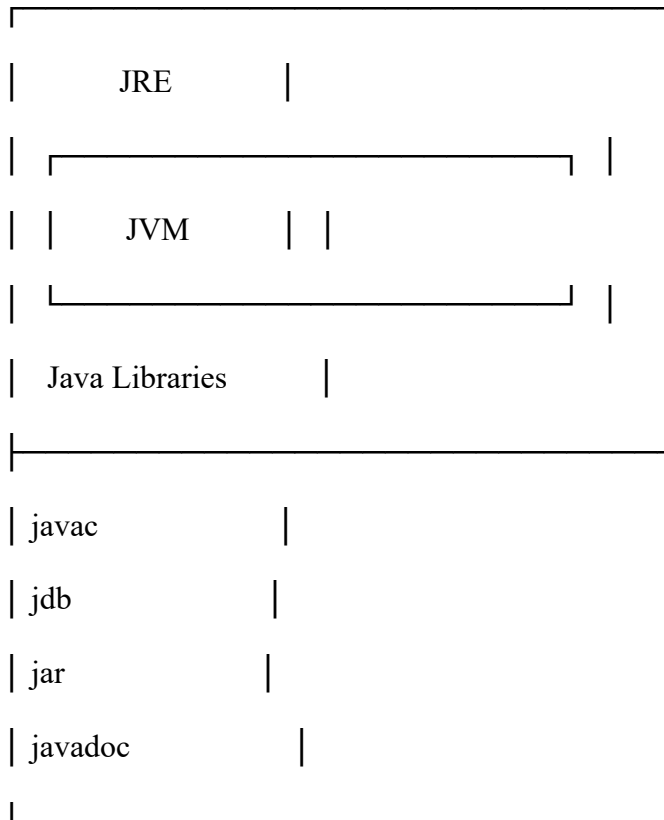
11. Java Development Kit (JDK)

Definition

JDK (Java Development Kit) is a complete software package used to develop, compile, debug, and run Java applications.

Diagram

JDK



Explanation

JDK contains:

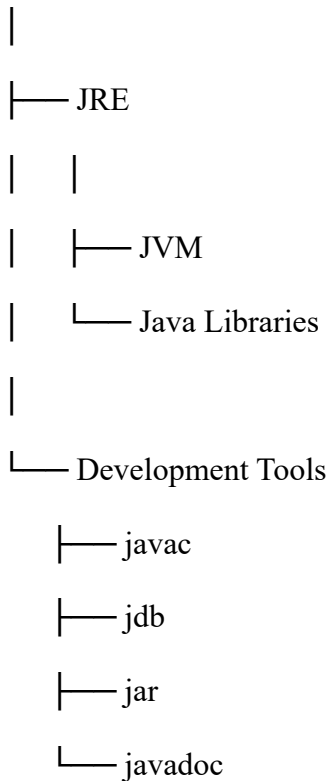
- JRE
- Java Compiler (javac)
- Debugger (jdb)
- Documentation Tool (javadoc)
- Archive Tool (jar)
- Other development utilities

Programmers install the JDK because it provides everything needed to write, compile, debug, and execute Java applications.

12. Relationship between JDK, JRE, and JVM

Diagram

JDK



Explanation

- **JDK** is the complete development package.
- **JRE** provides the runtime environment.
- **JVM** executes Java bytecode.
- Every JDK contains a JRE, and every JRE contains a JVM.

Difference between JDK, JRE and JVM

Feature	JDK	JRE	JVM
Full Form	Java Development Kit	Java Runtime Environment	Java Virtual Machine
Purpose	Develop and Run Java Programs	Run Java Programs	Execute Bytecode
Compiler	✓	✗	✗
JVM Included	✓	✓	Itself
Java Libraries	✓	✓	✗
Used By	Programmers	End Users	Runtime Environment

Real-Life Analogy

Imagine building and driving a car.

- **JDK** → Complete car manufacturing factory (tools to build the car).
- **JRE** → Finished car ready to drive.
- **JVM** → Engine that makes the car run.

Without the engine (JVM), the car cannot move.

13.Execution Flow of Java Program

Diagram

Java Source Code (.java)

|



Java Compiler (javac)

|



Bytecode (.class)

|



Class Loader

|



Bytecode Verifier

|



Interpreter / JIT

|



Machine Code

|



Operating System



Hardware

Explanation

1. The programmer writes the source code in a .java file.
2. The Java Compiler (javac) converts it into bytecode (.class file).
3. The Class Loader loads the bytecode into memory.
4. The Bytecode Verifier checks that the bytecode is secure and valid.
5. The Interpreter and JIT Compiler convert the bytecode into machine code.
6. The Operating System executes the machine code.
7. The Hardware performs the required operations and displays the output.

Exam Summary

- **JDK = JRE + Development Tools**
- **JRE = JVM + Java Libraries + Runtime Files**
- **JVM = Executes Java Bytecode**
- **Compiler (javac) = Converts .java to .class**
- **Bytecode = Platform-independent intermediate code**
- **JIT = Improves execution speed**
- **Garbage Collector = Automatically frees unused memory**
- Java offers platform independence, security, robustness, portability, multithreading, and distributed computing.
- **Bytecode** is the intermediate code generated by the Java compiler.
- The **JVM** executes bytecode and converts it into machine code.
- The **JRE** provides the environment required to run Java programs.
- The **JDK** provides all the tools needed to develop, compile, and run Java applications.
- **Relationship: $\text{JDK} \supset \text{JRE} \supset \text{JVM}$.**

Lecture – 3 (1 Hour)

Java Environment Setup, Java Program Structure, Procedure-Oriented Programming (POP) vs Object-Oriented Programming (OOP)

Learning Outcomes

- Understand the Java development environment.
 - Install and configure Java on a computer.
 - Write and understand the structure of a Java program.
 - Explain the difference between Procedure-Oriented Programming (POP) and Object-Oriented Programming (OOP).
 - Understand why Java follows the OOP approach.
-

3.1 Java Environment Setup

Before writing Java programs, we need to install the required software on the computer. This process is known as **Java Environment Setup**.

Requirements for Java Programming

To develop Java applications, we need:

- Java Development Kit (JDK)
- Text Editor or IDE
- Command Prompt (Terminal)

Popular IDEs

- IntelliJ IDEA
- Eclipse
- NetBeans
- Visual Studio Code

Note: Beginners can also write Java programs using **Notepad** and compile them using the **Command Prompt**.

Step 1: Download JDK

Download the latest **Java Development Kit (JDK)** from Oracle's official website.

After downloading:

- Run the installer.
- Click **Next** until installation completes.
- Note the installation path.

Example:

C:\Program Files\Java\jdk-21

Step 2: Set the PATH Environment Variable

Setting the **PATH** allows the operating system to recognize Java commands from any folder.

Steps

1. Open **System Properties**.
2. Click **Environment Variables**.
3. Select **Path** under System Variables.
4. Click **Edit**.
5. Add the JDK's **bin** folder.

Example:

C:\Program Files\Java\jdk-21\bin

Step 3: Verify Installation

Open **Command Prompt** and type:

```
java -version
```

Sample Output

```
java version "21"
```

Now check the compiler:

```
javac -version
```

Sample Output

```
javac 21
```

If both commands work, Java has been installed successfully.

Java Development Process

Write Program (.java)

|



Compile using javac

|



Bytecode (.class)

|



Run using java

|



Output

3.2 Java Program Structure

Every Java program follows a fixed structure.

Basic Structure

```
// Sample Java Program
```

```
class Hello {  
  
    public static void main(String[] args) {  
  
        System.out.println("Hello Java");  
  
    }  
  
}
```

Explanation of Each Part

1. Comment

Comments improve program readability.

Single-line Comment

```
// This is a comment
```

Multi-line Comment

```
/*
```

This is

a multi-line comment

```
*/
```

Comments are ignored by the compiler.

2. Class Declaration

Every Java program is written inside a **class**.

Syntax:

```
class ClassName
```

```
{  
  
}
```

Example:

```
class Student
```

```
{  
  
  
}
```

3. Main Method

The main() method is the entry point of every Java application.

Syntax:

```
public static void main(String[] args)
```

Explanation

Keyword	Meaning
public	Accessible from anywhere
static	Can be called without creating an object
void	Does not return any value
main	Starting point of program execution
String[] args	Stores command-line arguments

4. Statements

Statements perform actions.

Example:

```
System.out.println("Welcome");
```

Each statement ends with a semicolon (;).

5. Output Statement

Java uses:

```
System.out.println();
```

to display output.

Example:

```
System.out.println("Apollo Institute");
```

Output:

Apollo Institute

Complete Program Example

```
// Program to print student details
```

```
class Student {
```

```
    public static void main(String[] args) {
```

```
        System.out.println("Name : Rahul");
```

```
        System.out.println("Course : BCA");
```

```
        System.out.println("Semester : 3");
```

}

}

Output

Name : Rahul

Course : BCA

Semester : 3

Java Program Execution Process

Student.java

|



javac Student.java

|



Student.class

|



java Student

|



Program Output

Rules for Writing Java Programs

- Class names should begin with a capital letter (recommended).
 - File name and public class name must be the same.
 - Every statement ends with a semicolon.
 - Java is case-sensitive.
 - Curly braces {} define blocks of code.
-

Common Mistakes

Missing Semicolon

```
System.out.println("Hello")
```

Compiler Error

Wrong Class Name

File Name:

Hello.java

Class Name:

```
class Test
```

This causes an error if the class is declared public.

Incorrect Main Method

```
public void main()
```

Program will not execute because the JVM cannot find the correct entry point.

3.3 Procedure-Oriented Programming (POP)

Definition

Procedure-Oriented Programming (POP) is a programming approach where the program is divided into **functions (procedures)**. The focus is on performing a sequence of actions to solve a problem.

Examples of POP languages:

- C
 - Pascal
-

Characteristics of POP

- Program is divided into functions.
 - Data is shared among functions.
 - Functions operate on data.
 - Top-down approach.
 - Less secure because data can be accessed directly.
-

Real-Life Example of POP

Imagine a **restaurant kitchen**.

Different workers perform different tasks:

- One cuts vegetables.
- One prepares dough.
- One cooks food.

Each person performs a **function**, but everyone can access the same ingredients. Since the data (ingredients) is shared, mistakes or misuse can occur.

Advantages of POP

- Simple for small programs.
- Faster execution for simple tasks.

- Easy to understand basic logic.
-

Limitations of POP

- Difficult to maintain large programs.
 - Poor security.
 - No data hiding.
 - Code reusability is limited.
-

3.4 Object-Oriented Programming (OOP)

Definition

Object-Oriented Programming (OOP) is a programming approach where programs are built using **objects** and **classes**. Data and functions are combined into a single unit called an object.

Examples of OOP languages:

- Java
 - C++
 - Python
 - C#
-

Characteristics of OOP

- Uses classes and objects.
 - Supports data hiding.
 - Provides code reusability.
 - Follows a bottom-up approach.
 - More secure and easier to maintain.
-

Real-Life Example of OOP

Consider a **Car**.

A car has:

Data (Attributes)

- Color
- Model
- Speed

Functions (Methods)

- Start()
- Stop()
- Accelerate()

In OOP, the data and methods are grouped together inside the **Car** object.

POP vs OOP

Basis	POP	OOP
Full Form	Procedure-Oriented Programming	Object-Oriented Programming
Focus	Functions	Objects
Approach	Top-down	Bottom-up
Program Structure	Functions	Classes and Objects
Data Security	Less secure	More secure
Data Hiding	Not supported	Supported
Code Reusability	Limited	High
Maintenance	Difficult	Easier
Suitable For	Small programs	Large applications
Examples	C, Pascal	Java, C++, Python

Why Java Uses OOP?

Java follows the OOP approach because it provides:

- Better code organization.
- Code reusability.
- Improved security through encapsulation.
- Easier maintenance.
- Scalability for large applications.

POP vs OOP Diagram

POP

Program



Functions



Shared Data

OOP

Program



Classes



Objects



Data + Methods

Summary

- Java Environment Setup involves installing the **JDK**, configuring the **PATH**, and verifying the installation.
- A Java program consists of **comments**, **class**, **main() method**, and **statements**.
- The **main()** method is the starting point of every Java application.
- **POP** organizes programs into functions and is suitable for small applications.
- **OOP** organizes programs into classes and objects, making it more secure, reusable, and suitable for large software systems.
- Java uses the **Object-Oriented Programming** approach because it supports modularity, reusability, and data security.

Lecture – 4

OOP Concepts, Hello World Program, Compilation & Common Errors

Learning Outcomes

After completing this lecture, learners will be able to:

- Explain the basic concepts of Object-Oriented Programming (OOP).
 - Understand Classes, Objects, Superclass, and Subclass.
 - Differentiate between Method Overloading and Polymorphism.
 - Explain Message Communication in OOP.
 - Write, compile, and execute a simple Java program.
 - Identify common compilation and runtime errors.
-

4.1 Introduction to Object-Oriented Programming (OOP)

Definition

Object-Oriented Programming (OOP) is a programming approach in which programs are developed using **objects** and **classes**. It combines **data (attributes)** and **functions (methods)** into a single unit called an **object**.

Java is an **Object-Oriented Programming Language** because it follows OOP principles.

Why OOP?

Suppose a college management system is developed.

Instead of writing separate functions for students, teachers, and courses, OOP creates separate **objects** such as:

- Student
- Teacher
- Course
- Library

Each object stores its own data and methods.

This makes programs:

- Easier to maintain
- More secure
- Reusable
- Easy to expand

Basic Terminology

Class

Definition

A **Class** is a blueprint or template used to create objects.

It defines:

- Data (Variables)
- Functions (Methods)

Example

Think of a **House Plan**.

A house plan is not an actual house.

It is only a design.

Similarly,

A class is only a blueprint.

Object

Definition

An **Object** is an instance of a class.

Objects occupy memory and perform actions.

Example

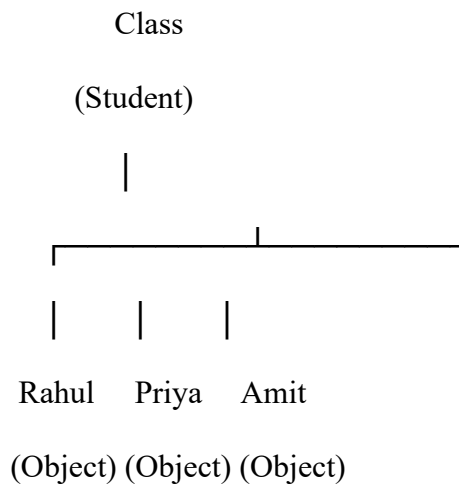
Class → Student

Objects →

- Rahul
- Priya
- Amit

Each student has different values but belongs to the same class.

Relationship



4.2 Abstraction

Definition

Abstraction is the process of hiding unnecessary implementation details and showing only the essential features to the user.

The user knows **what to do**, not **how it works internally**.

Real-Life Example

ATM Machine

When withdrawing money, the customer only:

- Inserts ATM card
- Enters PIN

- Selects amount

The customer does not know how the ATM communicates with the bank server.

This is **Abstraction**.

Another Example

Driving a car.

The driver only uses:

- Steering
- Brake
- Accelerator

The internal engine mechanism remains hidden.

Advantages

- Reduces complexity.
 - Improves security.
 - Makes software easier to use.
-

4.3 Encapsulation

Definition

Encapsulation is the process of combining data and methods into a single unit (class) and restricting direct access to data.

It is also called **Data Hiding**.

Real-Life Example

Capsule Medicine

A medicine capsule contains many ingredients inside.

The user only consumes the capsule.

Similarly,

In Java,

Data and methods are packed together inside a class.

ATM Example

Customers cannot directly access the bank database.

They interact only through ATM operations.

Advantages

- Data Security
 - Data Hiding
 - Better Control
 - Easy Maintenance
-

4.4 Inheritance

Definition

Inheritance is the process by which one class acquires the properties and methods of another class.

The existing class is called the **Superclass (Parent Class)**.

The new class is called the **Subclass (Child Class)**.

Real-Life Example

Father

↓

Son

The son inherits:

- Surname
- Family property
- Some characteristics

Similarly,

A child class inherits variables and methods from the parent class.

Advantages

- Code Reusability
- Easy Maintenance
- Faster Development

Superclass and Subclass

Superclass	Subclass
Parent Class	Child Class
Existing Class	Derived Class
Gives properties	Receives properties

4.5 Polymorphism

Definition

Polymorphism means "**many forms.**"

The same method performs different tasks depending on the object or situation.

Real-Life Example

Payment System

Suppose you pay shopping bills.

Payment can be made using:

- Cash
- UPI

- Credit Card
- Debit Card

The action is the same:

Pay

But the implementation is different.

This is **Polymorphism**.

Another Example

A person can be:

- Teacher in college
- Parent at home
- Customer in a supermarket

One person performs different roles.

Advantages

- Flexibility
 - Code Reusability
 - Easy Maintenance
-

4.6 Method Overloading

Definition

Method Overloading means defining multiple methods with the **same name** but **different parameter lists** in the same class.

It is an example of **Compile-Time Polymorphism**.

Example

```

class Calculator {

    int add(int a, int b) {

        return a + b;

    }

    int add(int a, int b, int c) {

        return a + b + c;

    }

}

```

Here, the method **add()** is overloaded because the number of parameters is different.

Real-Life Example

A mobile phone camera has multiple ways to take a photo:

- Photo mode
- Portrait mode
- Night mode

The action is the same (**Capture Photo**), but different settings are used.

Difference between Overloading and Polymorphism

Method Overloading	Polymorphism
Same method name	Same operation in different forms
Different parameters	Different implementation
Compile-Time	Compile-Time or Run-Time
Same class	Same or different classes

4.7 Message Communication

Definition

In Object-Oriented Programming, **objects communicate with each other by sending messages**.

A message is a request to perform a specific operation.

Real-Life Example

Teacher asks a student:

"Submit Assignment."

The teacher sends a message.

The student performs the requested task.

Similarly,

Object A calls a method of Object B.

Diagram

Object A

|

| Message



Object B

|

Executes Method

Advantages

- Easy communication
- Better modularity
- Code reusability

4.8 Hello World Program

The first Java program is traditionally called **Hello World**.

Program

```
class Hello {  
  
    public static void main(String[] args) {  
  
        System.out.println("Hello World");  
  
    }  
  
}
```

Explanation

class Hello

Defines a class named **Hello**.

public static void main(String[] args)

The entry point of the Java program.

System.out.println();

Displays output on the screen.

Output

Hello World

Steps to Compile and Run Java Program

Step 1

Write the program.

File Name:

Hello.java

Step 2

Compile

```
javac Hello.java
```

Output

Hello.class

Step 3

Run

```
java Hello
```

Program Execution Process

Hello.java

|



```
javac Hello.java
```

|



Hello.class

(Bytecode)



java Hello



JVM



Machine Code



Output

Common Errors in Java

Java errors are mainly classified into three categories:

1. Compile-Time Errors

These errors occur during compilation.

Example

Missing semicolon

```
System.out.println("Hello")
```

Compiler displays an error because the semicolon is missing.

Another example:

Misspelled keyword

```
publik class Hello
```

Compiler Error because **public** is misspelled.

2. Runtime Errors

These occur while the program is running.

Example

```
int a = 10;
```

```
int b = 0;
```

```
System.out.println(a / b);
```

Output:

ArithmeticException

The program compiles successfully but fails during execution.

3. Logical Errors

These errors do not produce compiler messages.

The program runs successfully but gives the wrong result.

Example

```
int area = length + width;
```

Correct formula:

```
int area = length * width;
```

Tips to Avoid Errors

- Always end statements with a semicolon (;).
- Match opening and closing braces {}.
- Use the correct spelling of Java keywords.
- Save the file with the correct .java extension.
- If the class is public, the file name must match the class name.
- Carefully check program logic before execution.

Summary

- A **Class** is a blueprint, and an **Object** is its instance.
- **Abstraction** hides implementation details and exposes only essential features.
- **Encapsulation** binds data and methods together while protecting data.
- **Inheritance** enables a child class to acquire properties and methods from a parent class.
- **Polymorphism** allows the same action to have different implementations.
- **Method Overloading** means defining multiple methods with the same name but different parameters.
- **Message Communication** enables objects to interact by calling methods.
- A Java program is compiled using javac and executed using java.
- Java errors are categorized as **Compile-Time**, **Runtime**, and **Logical Errors**.

